

Unidade Lógica Aritmética

Analuz Lima da Silva, Larissa Matos Barbosa, Vivian Martins Moura

Departamento de Tecnologia - Universidade Estadual de Feira de Santana (UEFS)
44036-900 - Feira de Santana - BA - Brasil

Resumo. *Este trabalho descreve o desenvolvimento de uma Unidade Lógica e Aritmética (ULA) capaz de realizar operações lógicas e aritméticas na placa programável DE10-Lite. O funcionamento de cada operação é explicado com base na organização dos módulos utilizados para produção do código em Verilog e o auxílio de elementos gráficos como tabelas-verdade e imagens de circuitos, essenciais para a análise e estruturação da lógica. Além disso, os resultados evidenciam o bom desempenho da ULA e o cumprimento de todos os objetivos propostos.*

1. Introdução

A Unidade Lógica e Aritmética (ULA) é um circuito que constitui a base do fluxo de dados de diversos microprocessadores. Originada como um componente fundamental em sistemas digitais, a ULA é responsável por executar operações matemáticas e funções. A sua complexidade e a velocidade são proporcionais ao sistema em que será utilizada, variando desde calculadoras eletrônicas simples a computadores de grande porte [1].

Nesse contexto, a disciplina de Mi - Projeto de Circuitos Digitais propôs o desafio de projetar e implementar uma ULA. O objetivo principal é desenvolver um circuito que receba dois operandos de 4 bits e um sinal de entrada *Carry In*, que será somado ao bit menos significativo do primeiro operando. O sistema deve permitir a seleção de operações por meio de chaves na placa FPGA DE10-Lite, incluindo as aritméticas (soma, subtração, multiplicação e divisão) e as lógicas (*AND*, *OR* e *XOR*). Por fim, os resultados das operações devem ser exibidos em displays de 7 segmentos, enquanto o estado do processamento será indicado por quatro *flags* - *Carry Out*, *Overflow*, *Zero* e *Erro* - exibidas em LEDs.

Este relatório detalha o desenvolvimento da ULA, explicando a lógica e a implementação de cada módulo. A metodologia aborda os requisitos, a estrutura modular do projeto e as ferramentas utilizadas para a simulação e implementação. Por fim, a seção de resultados e discussões apresenta o manual de uso da placa DE10-Lite, os testes realizados para validação e os resultados obtidos.

2. Metodologia

A Unidade Lógica e Aritmética (ULA) foi desenvolvida por meio da Linguagem de Descrição de Hardware Verilog, seguindo os requisitos definidos pelo problema. Além disso, para melhor organização e otimização do código, o projeto foi dividido em módulos, que, ao realizar funções específicas, possibilitam que as operações lógicas e

aritméticas sejam realizadas. Nesse viés, esta seção discorre acerca dos requisitos, módulos e ferramentas utilizadas no desenvolvimento da ULA.

2.1. Definição dos Requisitos

Entradas

- Dois operandos de 4 bits cada;
- Um sinal de *Carry In* (Cin), proveniente de operações anteriores;
- Linhas de seleção de operação, controladas por chaves da placa DE10-Lite.

Saídas

- Os resultados da operação selecionada são exibidos:
 - Nos três displays de 7 segmentos (saída em representação decimal);
 - Nos LEDs da FPGA.
- Sinais de *flags* de status, indicando condições especiais do resultado.

Funcionalidades Especificadas

1. **Operações Aritméticas**
 - Soma
 - Subtração
 - Multiplicação
 - Divisão inteira
2. **Operações Lógicas**
 - *AND*
 - *OR*
 - *XOR*
3. **Geração de *Flags***
 - *Overflow*
 - *Zero*
 - *Carry Out*
 - *Erro*

2.2. Módulos

A Unidade Lógica e Aritmética (ULA) é uma parte importante de qualquer sistema de processamento de dados, e sua funcionalidade é definida pelos módulos que a compõem. Nesta seção, aprofundaremos na arquitetura e no funcionamento dos 17 módulos que foram implementados para conferir à ULA sua capacidade de processamento. Cada módulo, como o meio-somador, o somador completo e o multiplicador, representa um circuito especializado projetado para realizar uma operação lógica ou aritmética específica. A compreensão de como esses módulos individuais

operam, e como se combinam, é fundamental para entender a potência e a versatilidade da ULA, que pode executar uma ampla gama de tarefas computacionais.

2.2.1. Meio Somador

O *Meio Somador* é um circuito projetado para realizar a soma de dois bits únicos, resultando em uma saída de soma (S) e uma saída de transporte (*Carry Out*). A saída de soma representa o bit menos significativo do resultado, enquanto o *Carry Out* representa o "vai um", que é transferido para a próxima ordem de bits em uma operação de soma mais complexa.

Esse circuito é repetidamente usado nos demais módulos que serão apresentados a seguir. Sua estrutura é simples, e está apresentada na Figura 1 abaixo.

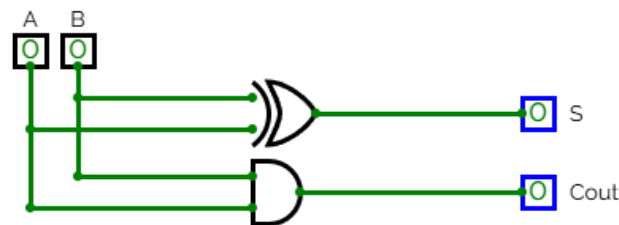


Figura 1. Circuito do Meio Somador

Abaixo, a Tabela-Verdade (Tabela 1) descreve o comportamento do Meio Somador para todas as combinações de entradas possíveis.

Tabela 1. Tabela Verdade do Meio Somador

A	B	S	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

2.2.2. Somador Completo

O Somador Completo é um circuito lógico que realiza a soma de três bits de entrada: A, B e um *Carry In* (Cin). A sua saída consiste em um bit de Soma (S) e um bit de *Carry Out* (Cout), que serve como entrada para o próximo estágio de soma em operações com múltiplos bits.

Este módulo é a base para a construção de circuitos somadores mais complexos, por isso ele foi repetidamente utilizado em diversos outros módulos desse projeto. A Figura 2 apresenta a imagem do circuito somador completo e a Tabela 2 demonstra o seu comportamento para todas as combinações de entradas possíveis.

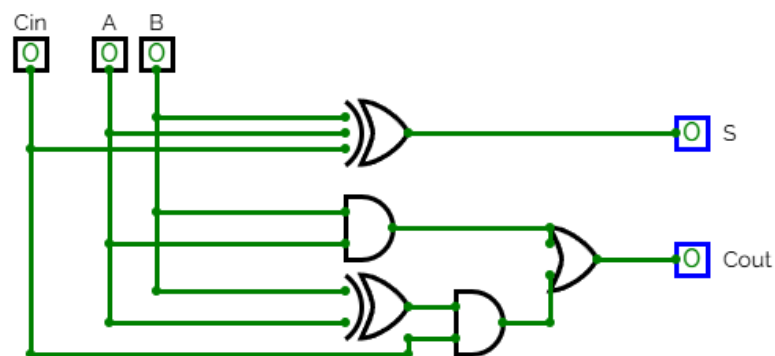


Figura 2. Circuito do Somador Completo

Tabela 2. Tabela Verdade do Somador Completo

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.2.3. Lógica do Carry In

Foi implementado um módulo com a função exclusiva de somar o *Carry In* ao bit menos significativo do operando *A*, cumprindo um dos requisitos propostos pelo desafio.

Para realizar essa operação, foram utilizados quatro módulos *Meio Somador* conectados em cascata (Figura 3). Essa organização garante que o *Carry Out* de cada somador seja propagado para o somador seguinte, permitindo que a soma do *Carry In* se estenda por todos os 4 bits da entrada *A*, se necessário. A saída final deste circuito, um barramento de 5 bits, torna-se o novo valor da entrada *A* que será utilizado em todas as operações subsequentes da ULA, sejam elas aritméticas ou lógicas.

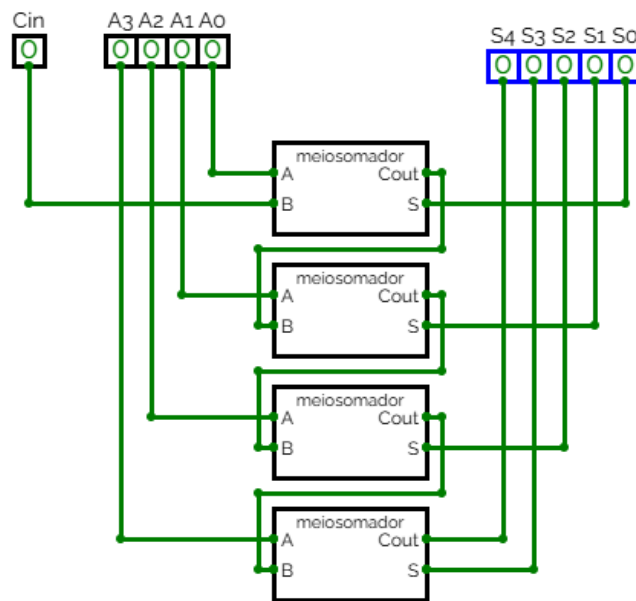


Figura 3. Circuito que Soma A + Carry In

Abaixo, a Tabela-Verdade (Tabela 3) descreve o comportamento do módulo $A + \text{Carry_In}$ para todas as combinações de entradas possíveis.

Tabela 3. Tabela Verdade da Soma de A + Carry In

Cin	A3	A2	A1	A0	S4	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0
0	0	0	1	1	0	0	0	1	1
0	0	1	0	0	0	0	1	0	0
0	0	1	0	1	0	0	1	0	1
0	0	1	1	0	0	0	1	1	0
0	0	1	1	1	0	0	1	1	1
0	1	0	0	0	0	1	0	0	0
0	1	0	0	1	0	1	0	0	1
0	1	0	1	0	0	1	0	1	0
0	1	0	1	1	0	1	0	1	1
0	1	1	0	0	0	1	1	0	0
0	1	1	0	1	0	1	1	0	1
0	1	1	1	0	0	1	1	1	0
0	1	1	1	1	0	1	1	1	1
1	0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	0	1	0

1	0	0	1	0	0	0	0	1	1
1	0	0	1	1	0	0	1	0	0
1	0	1	0	0	0	0	1	0	1
1	0	1	0	1	0	0	1	1	0
1	0	1	1	0	0	0	1	1	1
1	0	1	1	1	0	1	0	0	0
1	1	0	0	0	0	1	0	0	1
1	1	0	0	1	0	1	0	1	0
1	1	0	1	0	0	1	0	1	1
1	1	0	1	1	0	1	1	0	0
1	1	1	0	0	0	1	1	0	1
1	1	1	0	1	0	1	1	1	0
1	1	1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	0	0	0

2.2.4. Somador/Subtrator de 4 bits

Esse módulo é responsável pelas operações de soma e subtração. Ele foi projetado para operar com uma entrada *A* de 5 bits (resultado do módulo *A + Carry In*) e uma entrada *B* de 4 bits. A seleção entre soma e subtração é controlada por um sinal de operação (*Op*).

Para a operação de subtração, o circuito utiliza a técnica de complemento de dois [2]. Quando o sinal *Op* é definido como 1 (subtração), cada bit da entrada *B* passa por uma porta *XOR*, que atua como um inversor. Ao mesmo tempo, o sinal *Op* é injetado como *Carry In* no primeiro *Somador Completo* da cascata. Essa ação de inverter os bits e somar 1 é a definição do complemento de dois, o que permite que o mesmo circuito de soma execute a subtração.

O circuito, conforme ilustrado na Figura 4, foi construído com *Somadores Completos* em cascata, onde o *Carry Out* de um estágio se conecta ao *Carry In* do estágio seguinte.

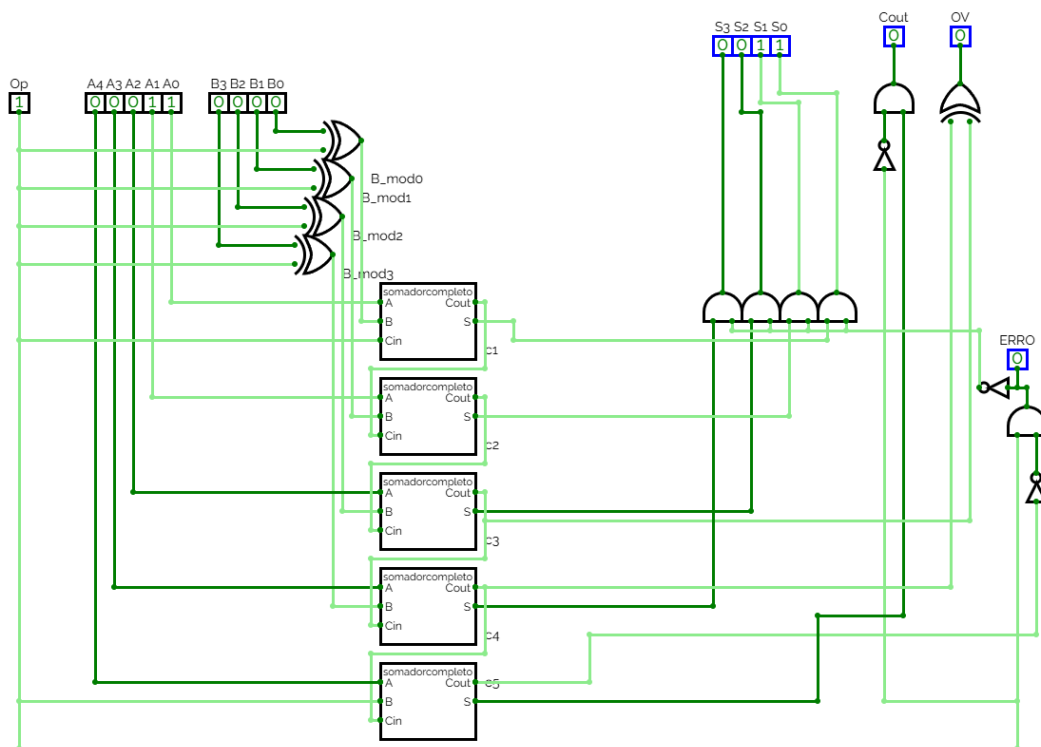


Figura 4. Circuito Somador Subtrator de 4 Bits

As três *flags* (*Overflow*, *Carry Out* e *Erro*) fornecem informações cruciais sobre o resultado da operação:

- **Overflow** (OV): Esta *flag* indica um erro de estouro em operações com sinal, ou seja, quando o resultado excede a faixa de representação de 4 bits em complemento de dois (-8 a +7). Isso ocorre quando a soma de dois números de mesmo sinal resulta em um número de sinal oposto. A maneira de detectar essa condição é verificar se o *Carry In* do 4º somador é diferente do *Carry Out* (também do 4º somador). Caso isso aconteça, significa que um “estouro” ocorreu. Uma porta *XOR* realiza essa comparação, pois sua saída é 1 apenas quando as entradas são diferentes.
- **Carry Out** (Cout): Esta *flag*, além de representar um estouro, funciona como o 5º bit de saída, indicando o resultado final da operação. O *Cout* do circuito está diretamente ligado à saída *S* do 5º Somador Completo, pois, tecnicamente, a operação é de 4 bits, e qualquer bit adicional é considerado um "estouro". Essa saída *S* é conectada a uma porta *AND* juntamente com o *Op* negado. Isso ocorre porque o *Cout* só é relevante em operações de soma, já que na subtração ele tem um significado diferente (indicando um "empréstimo"). Consequentemente, o *Carry Out* final será a saída dessa porta *AND*.
- **ERRO**: A *flag* de erro foi projetada para ser ativada especificamente quando uma operação de subtração resulta em um número negativo (ou seja, quando $B > A$). Em uma subtração via complemento de dois, se o resultado for negativo, o *Carry Out* final do circuito (neste caso, o *Cout* do 5º somador) será 0. Se o resultado

for positivo ou zero, o *Carry Out* será 1. Portanto, a condição de erro ($B > A$ na subtração) ocorre quando $Op = 1$ e o *Cout* do 5º somador é 0. A porta *AND* que conecta o sinal de operação (*Op*) e o *Cout* negado do 5º somador implementa exatamente essa lógica.

Devido à quantidade de possibilidades que uma tabela verdade com 10 entradas possui, o que resultaria em 1024 combinações, decidiu-se por não a incluir diretamente neste relatório. No entanto, a planilha com a tabela completa pode ser acessada através do botão a seguir:  Tabelas Verdade

2.2.5. Multiplicador

O módulo Multiplicador foi desenvolvido para realizar a multiplicação de dois números binários, sendo o operando *A* de 5 bits (resultado do módulo $A + \text{Carry In}$) e o operando *B* de 4 bits. Como o maior resultado possível é a multiplicação de 16 por 15 ($10000 * 1111$), que resulta em 240 (11110000), a saída do circuito foi projetada com 8 bits para atender todos os resultados.

A lógica de implementação deste circuito, baseada no conceito de *array multiplier* [3], pode ser dividida em duas etapas principais:

1. **Geração de Produtos Parciais:** Primeiramente, o circuito calcula os produtos parciais. Cada bit do operando *A* é multiplicado por cada bit do operando *B* através de um conjunto de portas *AND*. Esse processo é repetido para todas as combinações de bits das duas entradas, gerando um total de 20 produtos parciais.
2. **Soma dos Produtos Parciais:** Em seguida, esses produtos parciais são somados de forma organizada para obter o resultado final. Essa soma é realizada por uma estrutura em cascata de *Meio Somadores* e *Somadores Completos*, que adicionam os produtos parciais de maneira escalonada, simulando o método de multiplicação manual, onde cada linha de soma é deslocada para a esquerda. O bit menos significativo do resultado é simplesmente o primeiro produto parcial. Os bits subsequentes são o resultado da soma dos produtos parciais correspondentes e dos *carries* gerados pelas somas anteriores.

A estrutura completa, que combina as portas *AND* com a cascata de somadores, está ilustrada na Figura 5.

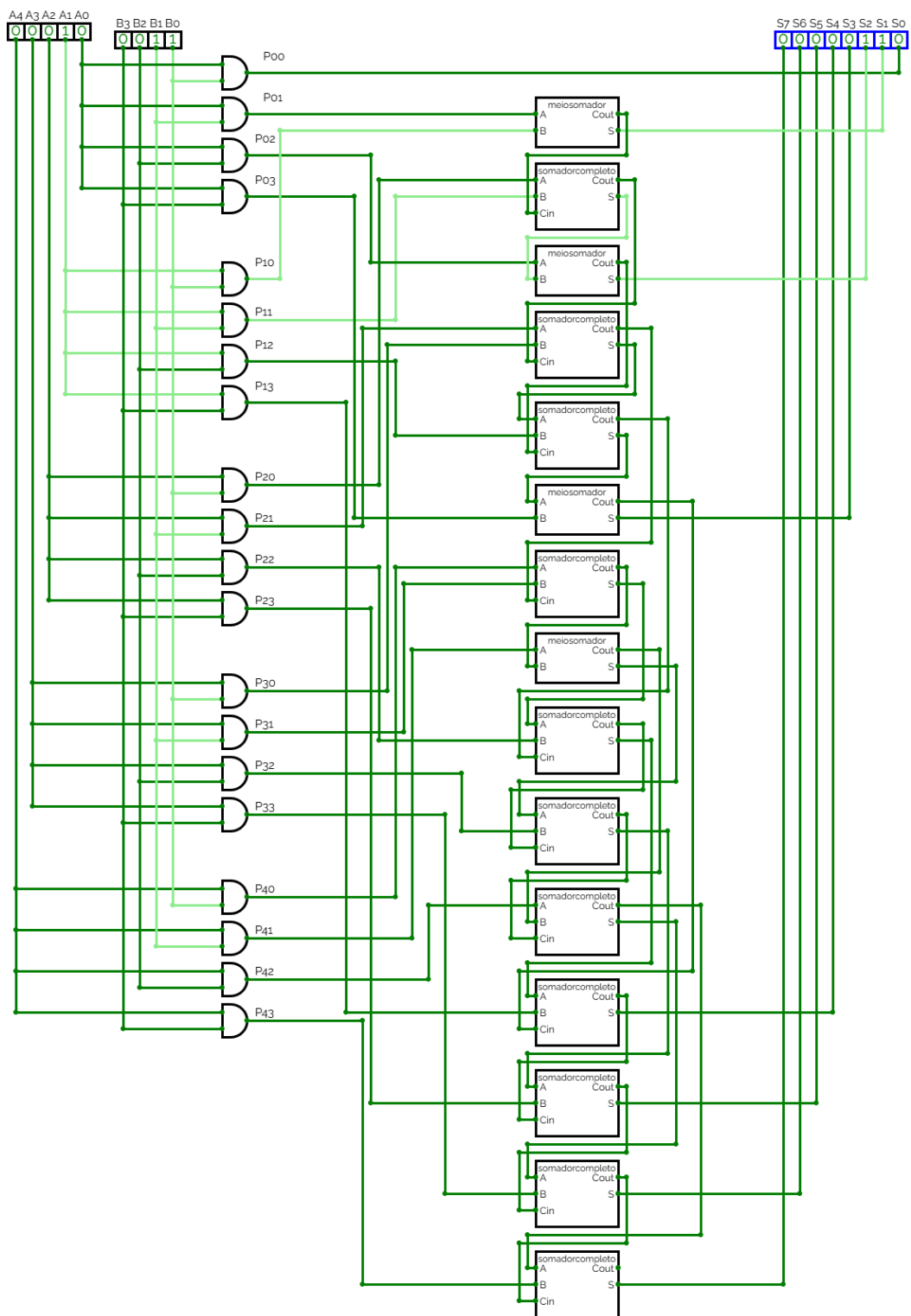


Figura 5. Circuito Multiplicador

Devido ao elevado número de entradas (9 bits), o que geraria uma tabela com 512 combinações, optou-se por não a incluir no relatório. No entanto, a planilha com a tabela completa pode ser acessada através do botão a seguir: [📄 Tabelas Verdade](#)

2.2.6. Divisor

O circuito *Divisor* é responsável pela realização da operação de divisão e foi produzido exclusivamente pela simplificação da sua tabela-verdade (disponível no link a seguir: [Tabelas Verdade](#)), com o auxílio da ferramenta de simulação Logisim.

Inicialmente, o circuito implementado para divisão funcionava como uma divisão comum, ou seja, os valores do dividendo parcial e do divisor eram comparados e a subtração era realizada caso o primeiro fosse maior que o segundo, utilizando, indiretamente, conceitos do Verilog sequencial, em que há o uso de memória e dos estados anteriores do circuito. Logo, devido à incompatibilidade entre os requisitos do problema e tal circuito, o *Divisor* foi implementado com as portas lógicas *AND*, *OR* e *NOT*, que transpõem as expressões booleanas geradas pela simplificação da tabela-verdade.

Ademais, este módulo é composto por três entradas, duas de quatro bits e uma de um bit (*carry-in*), e duas saídas, uma de quatro bits para o resultado da operação e uma de um bit para o erro, gerado sempre que o divisor é igual a zero. Vale ressaltar que cada bit do resultado é produzido por um submódulo conectado ao módulo principal, que disponibiliza o resultado final e o erro, caso exista.

2.2.7. AND

A porta *AND* é uma das mais básicas da lógica digital. Ela só gera saída igual a 1 quando todas as suas entradas também forem 1. No projeto a *AND* foi usada para implementar a operação lógica entre os operandos *A* e *B*, comparando bit a bit. Essa operação é bem simples, mas muito útil em circuitos digitais, já que serve para aplicar máscaras e verificar condições específicas. Sua estrutura é simples, e está apresentada na Figura 6 abaixo:

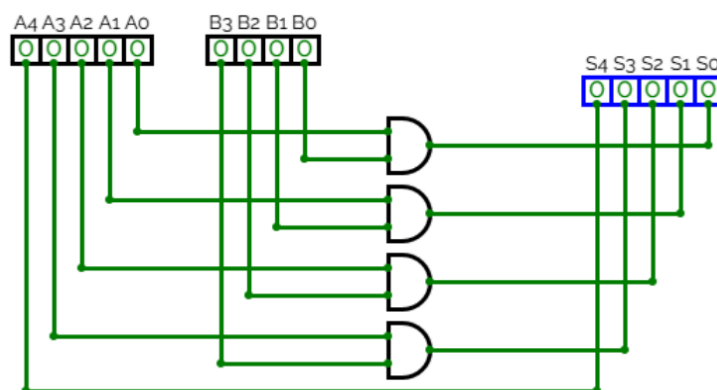


Figura 6. Circuito *AND*

Abaixo, o link para a Tabela-Verdade que descreve o comportamento do circuito *AND* para todas as combinações de entradas possíveis: [Tabelas Verdade](#)

2.2.8. OR

A porta *OR* funciona de forma parecida, mas a sua saída é 1 quando pelo menos uma das entradas é 1. Na ULA, ela foi responsável pela operação lógica *OR* entre os dois operandos. Esse tipo de operação aparece bastante em sistemas digitais quando é necessário “combinar” sinais ou quando basta uma das condições ser verdadeira. Sua estrutura é simples, e está apresentada na Figura 7 abaixo:

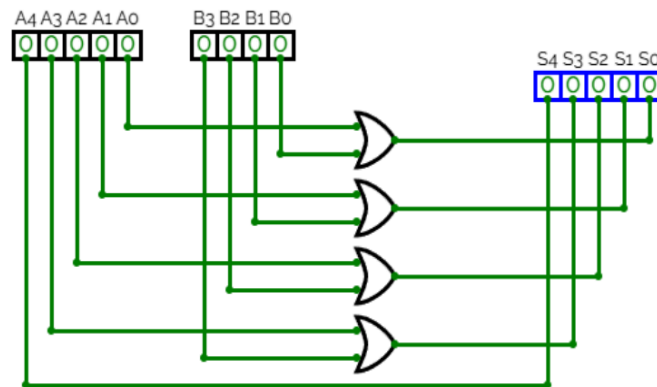


Figura 7. Circuito *OR*

Abaixo, o link para a Tabela-Verdade que descreve o comportamento do circuito *OR* para todas as combinações de entradas possíveis: [Tabelas Verdade](#)

2.2.9. XOR

A porta *XOR* (ou “OU exclusivo”) é um pouco diferente: ela só dá saída igual a 1 quando as entradas são diferentes entre si. No projeto, a *XOR* foi usada como mais uma das operações lógicas possíveis entre *A* e *B*. Ela é muito prática porque aparece em vários contextos, como na construção de somadores e até em circuitos de verificação de erros. Sua estrutura é simples, e está apresentada na Figura 8 abaixo:

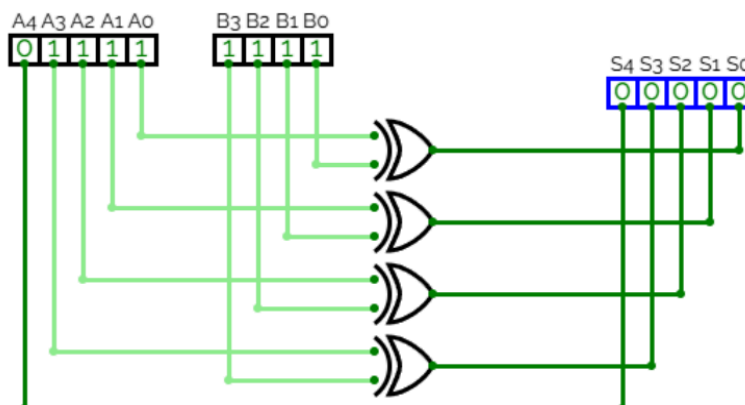


Figura 8. Circuito *XOR*

Abaixo, o link para a Tabela-Verdade que descreve o comportamento do circuito *XOR* para todas as combinações de entradas possíveis: [Tabelas Verdade](#)

2.2.10. Displays das Unidades e Dezenas

Os displays de 7 segmentos são dispositivos de saída que permitem a exibição de números decimais a partir de entradas binárias. Eles são formados por sete LEDs dispostos em formato de “8”, que podem ser acesos em diferentes combinações para representar os dígitos de 0 a 9. Cada segmento é identificado por uma letra de 'a' a 'g'. Para formar um número, o decodificador deve ativar a combinação correta desses LEDs. No caso desse projeto é utilizado um display do tipo ânodo comum, onde todos os ânodos (terminais positivos) dos LEDs são conectados a um ponto comum e é necessário aplicar um nível lógico baixo (GND) no terminal do segmento desejado para acendê-lo. Por exemplo, para formar o número 2, deve-se acender apenas os segmentos a,b,d,e,g. Como é mostrado na Figura 9:

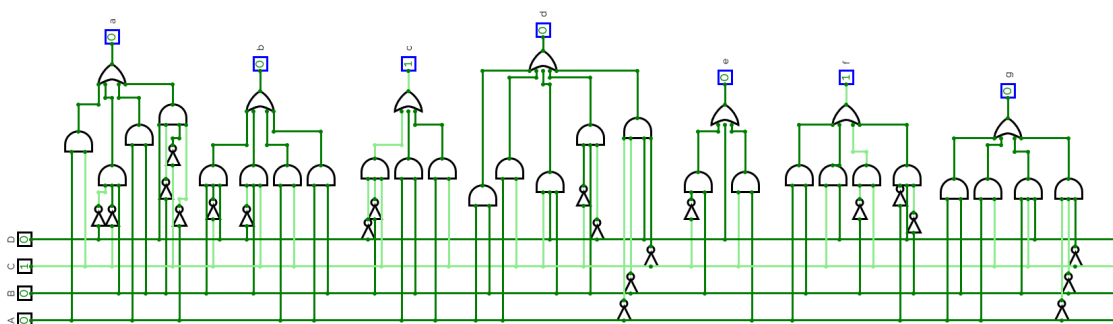


Figura 9. Circuito do display de 7 segmentos

Abaixo, o link para a Tabela-Verdade que descreve o comportamento do circuito do display para todas as combinações de entradas possíveis: [+ Tabelas Verdade](#)

2.2.11. Display das Centenas

O display das centenas tem como função exibir em decimal as centenas do resultado. Desse modo, como este valor pode ser no máximo dois - já que o maior resultado possível é 240 - foi produzido um circuito exclusivo para esse display, buscando-se reduzir a quantidade de elementos lógicos totais. Assim, a análise da tabela-verdade (Tabela 4) gerou o circuito ilustrado na Figura 10.

Tabela 4. Tabela-verdade do display das centenas

DÍGITO	A1	A0	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	1
1	0	1	1	0	0	1	1	1	1
2	1	0	0	0	1	0	0	1	0

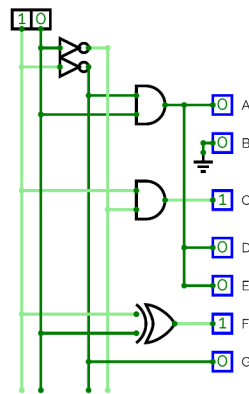


Figura 10. Circuito do display das centenas

2.2.12. Multiplexador de 2 para 1

O multiplexador 2 para 1 seleciona o nível lógico um ou zero como saída, dependendo do seletor e das entradas (A e B). Nessa perspectiva, a saída será igual a A, caso o seletor seja igual a zero, e igual a B, caso o seletor seja um, como exibe a Tabela 5. A partir dessa lógica, no módulo *Double Dabble*, este circuito é responsável por definir se o número será somado ou não com três, gerando-o em binário (11) para que a soma ocorra.

Tabela 5. Tabela-verdade do multiplexador 2 para 1

A	B	Sel	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

2.2.13. Double Dabble

Cada um dos sete subcircuitos **somador/seletor**, que foi feito baseado em um vídeo no youtube[5], opera em um conjunto de 4 bits do número binário de 8 bits de entrada. A função principal desses módulos é fazer uma verificação e, com base nela, realizar uma soma condicional. A verificação é simples: ele determina se o valor do grupo de 4 bits que está processando é maior ou igual a 5 (em decimal).

- **Se a condição for verdadeira** (o valor ≥ 5), o subcircuito ativa seu somador interno para adicionar 3 ao valor de entrada. Essa adição é crucial para ajustar o número binário e garantir que ele permaneça na faixa do BCD (0 a 9) após o deslocamento de bits.
- **Se a condição for falsa** (o valor < 5), o somador/seletor adiciona 0, o que significa que o valor de entrada é mantido sem alteração.

Essa lógica é essencial, pois ao processar um número binário de vários bits em etapas, cada grupo de 4 bits precisa ser ajustado individualmente para se manter dentro do formato BCD. O resultado da saída de um módulo se torna a entrada para o próximo, em cascata, até que o número inteiro seja totalmente convertido. Sua estrutura é simples e está representada na Figura 11:

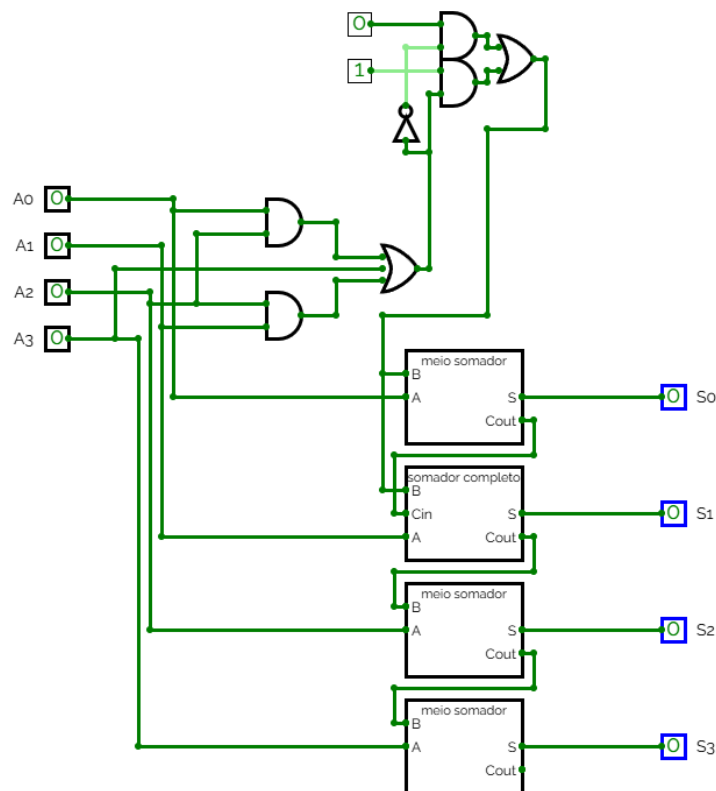


Figura 11. Circuito do subcircuito do decodificador

Abaixo, o link para a Tabela-Verdade que descreve o comportamento do circuito somador/seletor, subcircuito do decodificador, para todas as combinações de entradas possíveis: [Tabelas Verdade](#)

2.2.14. Multiplexador de 8 para 1

O multiplexador 8 para 1 funciona de maneira similar ao multiplexador 2 para 1, porém, possui oito entradas, selecionando uma delas como saída por meio do seletor de três bits, que possibilita a existência de um código específico para cada operação. Sob essa

lógica, o multiplexador receberá como entrada o resultado de cada operação e, a partir da combinação dos seletores, enviará como saída final o *S* da operação selecionada.

Dessa maneira, já que o resultado pode conter até oito bits, foram utilizados oito multiplexadores, sendo que quatro deles possuem como entradas os bits 1, 2, 3 e 4 da soma, subtração, multiplicação, divisão, *AND*, *OR* e *XOR*, um deles, os bits 4 da soma e multiplicação, e três deles, os bits 6, 7 e 8 da multiplicação. Vale ressaltar que, para as operações em que o resultado não ultrapassa quatro ou cinco bits (como a subtração que possui um resultado de, no máximo, 4 bits), os multiplexadores recebem zero como entrada, além disso, a sequência 111, que não possui uma operação atribuída, sempre terá como resultado zero.

2.2.15. Decodificador

O circuito apresentado é um decodificador binário de 8 bits para BCD, fundamental para a visualização de números em displays de 7 segmentos. Seu funcionamento baseia-se no método "*shift-and-add-3*", um algoritmo de conversão eficiente implementado através de uma cascata de subcircuitos somador/seletor. A lógica central do circuito é a seguinte: à medida que os bits do número binário de entrada são deslocados e processados, cada um dos sete módulos de somador/seletor avalia o valor do grupo de 4 bits em que opera.

- **Se o valor do grupo for maior ou igual a 5**, o circuito adiciona 3 (em binário) a esse grupo. Essa operação garante que o valor final se mantenha no formato BCD (0-9) após o deslocamento do bit seguinte.
- **Se o valor do grupo for menor que 5**, nenhuma alteração é feita; a soma é de 0, e o número prossegue inalterado para o próximo estágio.

Essa operação de soma condicional é repetida em cada estágio, garantindo que o número binário original seja traduzido corretamente para o formato BCD. O resultado final, que é a saída BCD, pode então ser conectado a um decodificador de 7 segmentos (como o que discutimos anteriormente) para exibir o número decimal correspondente no display. Sua estrutura é simples e está representada na Figura 12 abaixo:

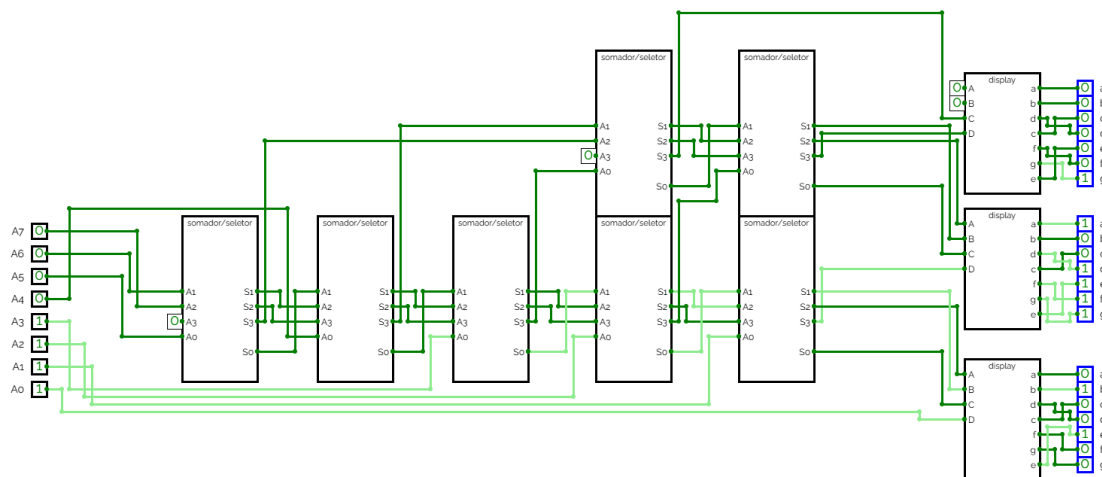


Figura 12. Circuito do decodificador

Abaixo, o link para a Tabela-Verdade que descreve o comportamento do circuito do decodificador para todas as combinações de entradas possíveis: [Tabelas Verdade](#)

2.2.16. Display das Operações

Como uma implementação extra, foi desenvolvido um módulo dedicado a exibir um símbolo correspondente à operação selecionada em um display de 7 segmentos. Este recurso fornece um *feedback* mais visual da operação, permitindo que o usuário identifique facilmente qual função a ULA está executando no momento.

O funcionamento do circuito é direto: ele recebe como entrada os três bits do seletor de operações (SEL) e, para cada uma das oito combinações possíveis, ativa os segmentos necessários para formar um caractere ou símbolo único que representa a operação. A Figura 13 ilustra os símbolos criados para cada uma das operações.

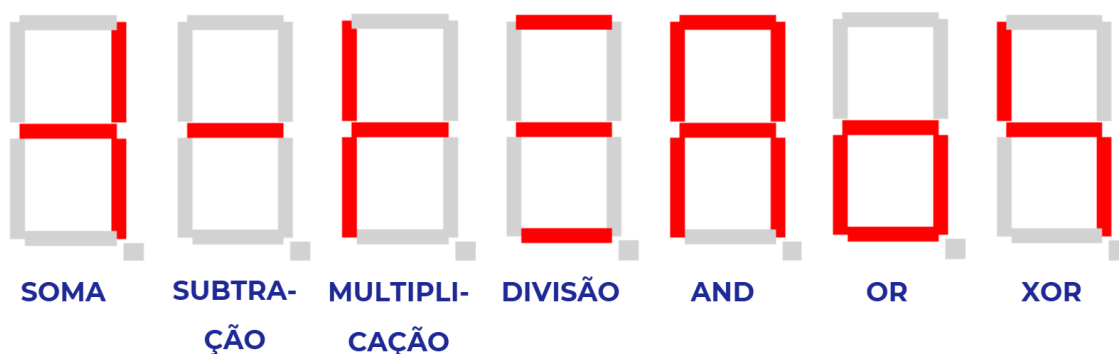


Figura 13. Símbolos das Operações Lógicas e Aritméticas

A Tabela 6, a seguir, descreve o comportamento do módulo, mostrando a relação entre a entrada do seletor e a saída para o display.

Tabela 6. Tabela Verdade dos Displays das Operações

	SEL2	SEL1	SEL0	a	b	c	d	e	f	g
SOMA	0	0	0	1	0	0	1	1	1	0
SUBTR	0	0	1	1	1	1	1	1	1	0
MULTI	0	1	0	1	1	1	1	0	0	0
DIVIS	0	1	1	0	1	1	0	1	1	0
AND	1	0	0	0	0	0	1	0	0	0
OR	1	0	1	1	1	0	0	0	1	0
XOR	1	1	0	1	1	0	1	1	0	0
ERRO	1	1	1	1	1	1	1	1	1	1

2.2.17. ULA

A Unidade Lógica e Aritmética (ULA) é o módulo central que integra todas as operações desenvolvidas e gerencia as saídas para o usuário. Sua arquitetura foi projetada para processar todas as funções em paralelo e, em seguida, selecionar o resultado desejado.

- **Processamento de Entrada:** Inicialmente, as entradas *A* (4 bits) e o *Carry In* (1 bit) são processadas pelo módulo *soma_A_Cin*. O resultado, *a_mod* (5 bits), e a entrada *B* (4 bits) são então distribuídos simultaneamente para todos os módulos operacionais: Somador/Subtrator, Multiplicador, Divisor e as portas lógicas (*AND*, *OR*, *XOR*). Isso significa que todas as sete operações são calculadas ao mesmo tempo, independentemente da seleção do usuário.
- **Seleção da Operação:** Um multiplexador de 8 para 1 é o componente chave para a seleção do resultado. O sinal de 3 bits *SEL*, controlado pelas chaves da placa, determina qual das saídas dos módulos operacionais será enviada para o barramento de saída final. Cada uma das oito saídas do multiplexador corresponde a um bit do resultado final.
- **Gerenciamento de Saídas e Flags:** A saída do multiplexador passa por um sistema de controle final antes de se tornar a saída definitiva (*S*). Este sistema é governado principalmente pela *flag* de *ERRO*, que tem a maior prioridade sobre todas as outras saídas.
 - **ERRO:** A *flag* de *ERRO* é ativada em três cenários:
 - Na operação de subtração, se $B > A$.
 - Na operação de divisão, se $B = 0$.
 - Se o seletor for configurado com o valor 111, que é uma combinação inválida.
 - Quando a *flag* de *ERRO* é ativada, ela anula todas as outras saídas. Um habilitador de saída se torna o *ERRO* negado, e através de um conjunto de portas *AND*, força o barramento de saída *S* a ser 00000000 e desliga as *flags* *Overflow*, *Carry Out* e *Zero*.
 - **Overflow (OV) e Carry Out (Cout):** Estas *flags* são herdadas diretamente do módulo Somador/Subtrator. No entanto, elas só são habilitadas e podem apresentar valor 1 se a operação selecionada for soma ou subtração (*SEL*[2] e *SEL*[1] em 0) e se a *flag* de *ERRO* estiver inativa.
 - **Zero:** A *flag* *Zero* é ativada quando todos os 8 bits de saída final *S* são iguais a zero. Isso é verificado por uma porta *NOR*.
- **Exibição nos Displays:** Por fim, o resultado final de 8 bits do barramento *S* é enviado ao módulo decodificador, que o converte para o formato BCD (Codificação Binária Decimal) para ser exibido nos três displays de 7 segmentos. Um quarto display foi implementado para mostrar qual operação está sendo executada no momento, com base no valor do seletor.

2.3. Ferramentas Utilizadas

Para o desenvolvimento desta Unidade Lógica e Aritmética, as ferramentas utilizadas foram: o software de desenvolvimento e projetos para FPGAs (Field-Programmable Gate Array) Quartus Prime Lite Edition, na edição 23.1, e a HDL Verilog para construção do código, as plataformas de simulação de circuitos digitais CircuitVerse e Logisim para análise e teste dos circuitos, e o aplicativo de planilhas online Google Planilhas para elaboração das tabelas-verdade.

3. Resultados e Discussões

Nesta seção, serão apresentados os resultados práticos e a análise do funcionamento da ULA. O tópico abrange o manual de uso, que detalha a pinagem e a operação da ULA na placa DE10-Lite, até as entradas e saídas dados. Por fim, os resultados obtidos em cada teste são discutidos para comprovar a eficácia e o correto funcionamento do circuito projetado.

3.1. Manual de Uso

Este manual tem como objetivo orientar a utilização da Unidade Lógica e Aritmética (ULA) implementada na placa DE10-Lite. São descritas as funções de cada conjunto de pinos, a forma correta de inserção dos operandos, a seleção da operação desejada e a interpretação dos resultados apresentados nos LEDs e displays de 7 segmentos.

3.1.1. Entradas de Dados

Na Figura 14 abaixo observa-se a pinagem na placa DE10-Lite:

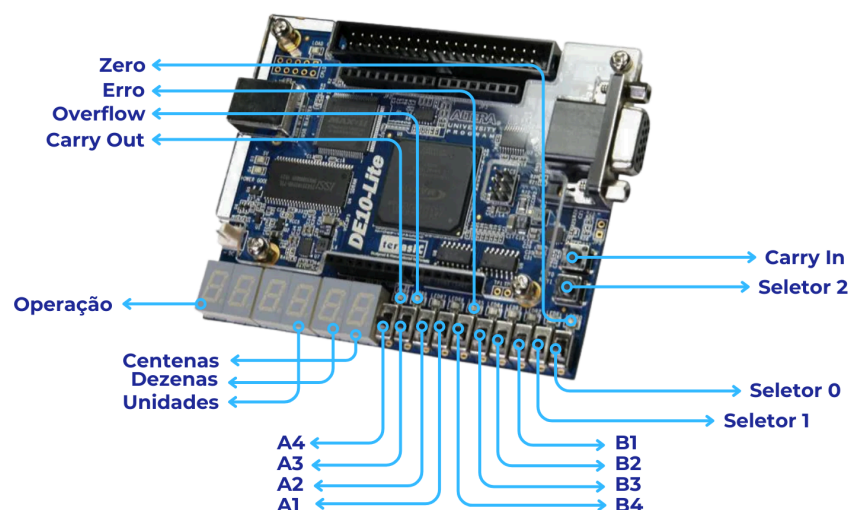


Figura 14. Pinagem na Placa DE10-Lite

- **Operandos A e B :**
 - O sistema recebe dois valores inteiros de 4 bits, correspondentes aos operandos das operações.

- As entradas estão distribuídas da seguinte forma:
 - Operando A $\rightarrow$ A1, A2, A3, A4
 - Operando B $\rightarrow$ B1, B2, B3, B4
- **Carry In (Cin):**
 - Entrada opcional para transporte de operações anteriores.
 - Ativado quando se deseja realizar somas encadeadas.

- **Seleção da Operação**

- A escolha da operação é realizada por meio de três sinais de controle (Seletor 0, Seletor 1 e Seletor 2), que juntos formam uma sequência de 3 bits. Na figura 15 abaixo é mostrada a tabela de seleção:

Seletor (S ₂ S ₁ S ₀)	Operação
000	Soma
001	Subtração
010	Multiplicação
011	Divisão
100	Divisão
100	AND
101	OR
110	XOR
111	Erro

Figura 15. Tabela de seleção

3.1.2. Saídas de Dados

- **Displays de 7 segmentos:**
 - Mostram o resultado em formato decimal, distribuído em unidades, dezenas e centenas, facilitando a leitura.
- **Sinais de Status (Flags)**
 - Quatro indicadores adicionais estão presentes para monitorar condições especiais:
 - **Zero (Z):** ativado quando o resultado da operação é 0.
 - **Erro (ERR):** ativado em situações inválidas, como divisão por zero.
 - **Overflow (OV):** sinaliza estouro de capacidade (quando o resultado excede 4 bits).
 - **Carry Out (COUT):** indica transporte gerado em operações aritméticas.

3.1.3. Procedimento de Utilização

1. Definir os operandos A e B nos respectivos pinos de entrada.
2. (Opcional) Desativar o *Carry In*, se desejado.
3. Selecionar a operação ajustando os seletores (S2, S1, S0) de acordo com a tabela de operações.
4. Observar o resultado:
 - Valor numérico nos displays de 7 segmentos.
5. Monitorar os sinais de status para identificar condições especiais (Zero, Erro, Overflow, Carry Out).

3.3. Testes e Resultados Obtidos

Durante o desenvolvimento deste projeto, diversos testes foram realizados para detecção e resolução de possíveis erros. Desse modo, após a elaboração de suas expressões booleanas, os circuitos de alguns módulos foram, gradualmente, construídos e testados no CircuitiVerse e Logisim, possibilitando que as saídas geradas nestas simulações pudessem ser comparadas com os valores das tabelas-verdade. Posteriormente, tais circuitos foram descritos em Verilog e testados novamente, mas, desta vez na placa DE10-Lite. Ademais, alguns módulos, como os multiplexadores e o display das operações, apenas foram testados por meio da ferramenta de simulação do Quartus e da própria placa.

Outrossim, após a construção e junção de todos os módulos, a ULA foi testada e foram identificados erros na exibição das *flags* de zero na multiplicação e de erro na subtração - quando a operação resultava em um número negativo. Entretanto, tais erros foram solucionados, garantindo a total integração entre os diferentes módulos e suas funcionalidades e o bom funcionamento da ULA na placa DE10-Lite.

4. Conclusão

O desenvolvimento da Unidade Lógica e Aritmética (ULA) de 4 bits permitiu atender plenamente a todos os objetivos propostos no enunciado do problema. O sistema implementado foi capaz de realizar corretamente as operações aritméticas (soma, subtração, multiplicação e divisão) e lógicas (*AND*, *OR* e *XOR*), além de contemplar o tratamento de situações especiais por meio das *flags* de status (*Zero*, *Overflow*, *Carry Out* e *Erro*).

Além de cumprir os requisitos estabelecidos, foram realizadas melhorias que ampliaram as funcionalidades da solução. Primeiramente, foi incluída a exibição do sinal da operação selecionada diretamente no display, o que proporciona maior clareza ao usuário durante a utilização do sistema. Em seguida, ao invés de apenas dois displays de 7 segmentos para apresentação do resultado, foram implementados três displays, possibilitando representar integralmente os resultados da multiplicação de 5×4 bits.

Dessa forma, o projeto não apenas alcançou os objetivos definidos, como também agregou recursos adicionais que aumentam a usabilidade do protótipo. Como sugestão de extensão futura, seria possível expandir a ULA para operandos de maior largura (8 ou 16 bits), além da implementação de operações adicionais, como potenciação por exemplo.

5. Referências

- [1] J.L.R.B.; P.S.C.; E.T.M. (2011) Unidade Lógica e Aritmética. *EPUSP PCS* 2011/2305/2355 Laboratório Digital. Disponível em: https://www2.pcs.usp.br/~labdig/pdffiles_2011/ula.pdf. Acesso em: 20 set 2025.
- [2] Messina, A.. (2019) Subtração binária em circuitos digitais. *TecDicas*. Disponível em: <https://tecnicas.com/subtracao-binaria-em-circuitos-digitais/>. Acesso em: 22 set 2025.
- [3] ELPROCUS. [s.d.] What is 4x4 Array Multiplier and Its Working. *Elprocus*. Disponível em: <https://www.elprocus.com/4x4-array-multiplier-and-its-working/>. Acesso em: 22 set 2025.
- [4] LIMA, Rogerio Antonio de. (2013) 14ª Aula de Eletrônica Digital - Decodificador BCD Display de 7 segmentos. *YouTube*. Disponível em: <https://www.youtube.com/watch?v=50AmTi2kWPo>. Acesso em: 23 set. 2025.
- [5] EATER, Ben. (2018) Double Dabble: Bin to BCD Conversion. *YouTube*. Disponível em: <https://youtu.be/yki0PkBcJbg>. Acesso em: 14 set. 2025.

6. Referências Consultadas

- DIEGUEZ, Daniel. [s.d.] Multiplicador de 4 Bits PDF. *Scribd*. Disponível em: <https://pt.scribd.com/document/402809914/multiplicador-de-4-bits-PDF>. Acesso em: 23 set. 2025.
- EPUSP. LABORATÓRIO DIGITAL. (2013) Projeto de um Divisor Binário. *EPUSP*. Disponível em: https://www2.pcs.usp.br/~labdig/pdffiles_2013/divisor-binario.pdf. Acesso em: 23 set. 2025.
- LIMA, Rogerio Antonio de. (2013) 14ª Aula de Eletrônica Digital - Decodificador BCD Display de 7 segmentos. *YouTube*. Disponível em: <https://www.youtube.com/watch?v=50AmTi2kWPo>. Acesso em: 23 set. 2025.
- LOGIC ALLIGATOR. (2021) What You Need To Know Before Building A Multi Digit Display - Double Dabble. *YouTube*. Disponível em:

- <https://www.youtube.com/watch?v=yki0PkBcJbg>. Acesso em: 23 set. 2025.
- MIDORIKAWA, Edson T. (2012) Multiplicador Binário com Sinal. São Paulo: EPUSP. Disponível em: https://www2.pcs.usp.br/~labdig/pdf/files_2012/multipl_sinal.pdf. Acesso em: 23 set. 2025.
- NESO ACADEMY. (2014) Shift Add 3 Method | Simple method for Binary to BCD. *YouTube*. Disponível em: <https://www.youtube.com/watch?v=IBgiB7KXfEY>. Acesso em: 23 set. 2025.
- TERASIC. (2016) DE10-Lite User Manual. Terasic Inc. Disponível em: https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/Boards/DE10-Lite/DE10_Lite_User_Manual.pdf. Acesso em: 23 set. 2025.
- TOCCI, Ronald J.; WIDMER, Neal S.; MOSS, Gregory L. (2016) Sistemas Digitais: princípios e aplicações. *Pearson Education do Brasil*. Disponível em: <https://archive.org/details/SistemasDigitais11EdicaoRonaldJTocci/mode/2up>. Acesso em: 23 set. 2025.
- ZUIM, Edgar. [s.d.] Somadores e Subtratores. Disponível em: https://www.ezuim.com/pdf/som_sub.pdf. Acesso em: 23 set. 2025.